



분석 함수

- 분석 함수는 데이터 그룹을 기반으로 앞뒤 행을 계산하거나 그룹에 대한 누적 분포, 상대 순위 등을 계산함
- 앞서 배운 집계 함수와 달리 분석 함수는 그룹마다 여러 행을 반환할 수 있음
- **앞 또는 뒤 행을 참조하는 함수 — LAG, LEAD**
 - 앞서 하루 전 날짜 데이터와 오늘 날짜 데이터를 비교할 때 SELF JOIN 문을 전일 대비 오늘의 수익을 구할 수 있었음
 - 이렇게 앞뒤 행을 비교하여 데이터 처리를 해야 하는 경우 LAG 또는 LEAD 함수를 사용하면 됨
 - 이 함수들을 사용하면 SELF JOIN 문을 따로 작성하지 않아도 돼 간편함

■ 앞 또는 뒤 행을 참조하는 함수 — LAG, LEAD

- LAG 함수는 현재 행에서 바로 앞의 행을,
LEAD 함수는 현재 행에서 바로 뒤의 행을 조회해 데이터를 처리함
- 물론 인자에 전달한 값에 따라 이전 또는 이후
몇 번째 행(데이터)를 참조할지 결정할 수도 있음

- LAG 함수, LEAD 함수의 기본 형식

LAG와 LEAD 함수의 기본 형식

LAG(또는 LEAD)(열 이름, 참조 위치)
OVER([PARTITION BY 열] ORDER BY 열)

- 인자로 참조 위치를 입력할 경우 기본값은 1이므로
아무것도 전달하지 않으면 1칸 앞이나 1칸 뒤의 데이터를 참조함

■ 앞 또는 뒤 행을 참조하는 함수 — LAG, LEAD

1. 다음은 payment 테이블에서 현재 행 기준으로 앞 또는 뒤의 행을 참조함.
여기서는 앞뒤 데이터를 쉽게 비교할 수 있도록
기준 열 amount를 가운데로 하여 결과를 출력함

Do it! LAG와 LEAD 함수로 앞뒤 행 참조

```
SELECT x.payment_date,  
       LAG(x.amount) OVER(ORDER BY x.payment_date ASC) AS  
lag_amount, amount,  
       LEAD(x.amount) OVER(ORDER BY x.payment_date ASC) AS  
lead_amount  
FROM (  
    SELECT date_format(payment_date, '%y-%m-%d') AS payment_date,  
    SUM(amount) AS amount  
    FROM payment GROUP BY date_format(payment_date, '%y-%m-%d')  
) AS x  
ORDER BY x.payment_date;
```

실행 결과

	payment_date	lag_amount	amount	lead_amount
▶	05-05-24	NULL	29.92	573.63
	05-05-25	29.92	573.63	754.26
	05-05-26	573.63	754.26	684.34
	05-05-27	754.26	684.34	804.04
	05-05-28	684.34	804.04	648.46
	05-05-29	804.04	648.46	628.42
	05-05-30	648.46	628.42	700.37
	05-05-31	628.42	700.37	57.84
	05-06-14	700.37	57.84	1376.52
	05-06-15	57.84	1376.52	1349.76

- 앞 또는 뒤 행을 참조하는 함수 — LAG, LEAD

1. 결과를 보면 amount 열 기준으로 왼쪽은 현재 행의 앞, 오른쪽은 현재 행의 뒤에 있는 행을 보여준다는 것을 알 수 있음.

첫 행인 29.92의 경우 앞의 값이 없으므로 NULL이 출력됨

■ 앞 또는 뒤 행을 참조하는 함수 — LAG, LEAD

2. 이번에는 LAG와 LEAD 함수의 인자로 2를 설정해 2행 앞이나 뒤를 참조하도록 작성해 보자

Do it! LAG와 LEAD 함수로 2칸씩 앞뒤 행 참조

```
SELECT x.payment_date,  
       LAG(x.amount, 2) OVER(ORDER BY x.payment_date ASC) AS lag_amount, amount,  
       LEAD(x.amount, 2) OVER(ORDER BY x.payment_date ASC) AS lead_amount  
FROM (  
    SELECT date_format(payment_date, '%y-%m-%d') AS payment_date,  
           SUM(amount) AS amount  
    FROM payment GROUP BY date_format(payment_date, '%y-%m-%d')  
  ) AS x  
ORDER BY x.payment_date;
```

■ 앞 또는 뒤 행을 참조하는 함수 — LAG, LEAD

2.

실행 결과				
	payment_date	lag_amount	amount	lead_amount
▶	05-05-24	NULL	29.92	754.26
	05-05-25	NULL	573.63	684.34
	05-05-26	29.92	754.26	804.04
	05-05-27	573.63	684.34	648.46
	05-05-28	754.26	804.04	628.42
	05-05-29	684.34	648.46	700.37
	05-05-30	804.04	628.42	57.84
	05-05-31	648.46	700.37	1376.52
	05-06-14	628.42	57.84	1349.76
	05-06-15	700.37	1376.52	1332.75

결과를 살펴보면 amount 열을 기준으로
lag_amount 열에서 반환하는 데이터는 이전 2행의 데이터이며,
lead_amount에서 반환하는 데이터는 이후 2행의 데이터임.

이렇게 LAG 또는 LEAD 함수를 사용하면 전후 데이터 차이 값 등을
자유롭게 연산할 때 편리하게 사용할 수 있음

▪ 누적 분포를 계산하는 함수 — CUME_DIST

- CUME_DIST 함수는 그룹 내에서 누적 분포를 계산함
- 다시 말해 그룹에서 데이터 값이 포함되는 위치의 누적 분포를 계산함
- CUME_DIST 함수의 기본 형식

CUME_DIST 함수의 기본 형식

```
CUME_DIST() OVER([PARTITION BY 열] ORDER BY 열)
```


▪ 누적 분포를 계산하는 함수 — CUME_DIST

- CUME_DIST 함수는 0 초과~1 이하 범위의 값을 반환하며, 같은 값은 항상 같은 누적 분포값으로 계산함
- 그동안 NULL은 정의되지 않은 값이라고 설명했지만 CUME_DIST 함수는 기본적으로 NULL값을 포함하며 NULL값은 해당 데이터 집합에서 가장 낮은 값으로 취급함

▪ 누적 분포를 계산하는 함수 — CUME_DIST

- 다음 쿼리를 통해 CUME_DIST 함수로 payment 테이블의 amount 열의 누적 분포값을 계산해 보자
- 쿼리를 살펴보면, customer_id 열의 데이터 그룹별로 amount 열의 데이터를 합산함
- 그리고 각 합산한 값에 CUME_DIST 함수를 사용하여 누적 분포를 조회함

▪ 누적 분포를 계산하는 함수 — CUME_DIST

Do it! CUME_DIST 함수로 누적 분포값 계산

```
SELECT x.customer_id, x.amount, CUME_DIST() OVER(ORDER BY
x.amount DESC)
FROM (
    SELECT customer_id, sum(amount) AS amount
    FROM payment GROUP BY customer_id
) AS X
ORDER BY x.amount DESC;
```

실행 결과

	customer_id	amount	CUME_DIST() OVER (ORDER BY x.amount DESC)
▶	526	221.55	0.001669449081803005
	148	216.54	0.00333889816360601
	144	195.58	0.005008347245409015
	137	194.61	0.008347245409015025
	178	194.61	0.008347245409015025
	459	186.62	0.01001669449081803
	469	177.60	0.011686143572621035
	468	175.61	0.0133559265442404
	236	175.58	0.015025041736227046
	181	174.66	0.01669449081803005
	176	173.63	0.018363939899833055
	50	169.65	0.02003338898163606

■ 누적 분포를 계산하는 함수 — CUME_DIST

- 결과를 살펴보면 amount 열의 데이터를 내림차순으로 정렬한 결과에 누적 분포를 계산함
- 그렇기 때문에 amount 열에서 가장 높은 customer=526의 amount 합계 값이 전체 데이터 중 상위 0.0016임
- 즉, 상위 0.16%의 누적 분포 값을 가지고 있으며, 그 다음 내림차순에 따라 누적 분포가 증가하는 값으로 마지막 행은 누적 분포 1을 반환하는 것을 확인할 수 있음

▪ 상대 순위를 계산하는 함수 — PERCENT_RANK

- PERCENT_RANK 함수는 지정한 그룹 또는 쿼리 결과로 이루어진 그룹 내의 상대 순위를 계산할 수 있음
- PERCENT_RANK 함수는 앞에서 배운 CUME_DIST 함수와 유사하지만 누적 분포가 아닌 분포 순위라는 점이 다름
- PERCENT 함수의 기본 형식

PERCENT_RANK 함수의 기본 형식

```
PERCENT_RANK() OVER([PARTITION BY 열] ORDER BY 열)
```

- 상대 순위를 계산하는 함수 — PERCENT_RANK
 - PERCENT_RANK 함수의 반환값 범위는 0 초과~1 이하임
 - 데이터 집합에서 첫 번째 행은 0값부터 시작하며 마지막 값은 1임
 - PERCENT_RANK 함수에서도 NULL은 해당 데이터 그룹에서 가장 낮은 값으로 취급되어, 상위 분포 순위를 계산할 때 하나의 데이터로 간주됨

■ 상대 순위를 계산하는 함수 — PERCENT_RANK

- 다음 쿼리를 통해 PERCENT_RANK 함수로 고객을 나타내는 customer_id 열의 데이터별로 결제한 총 금액을 합산해 이를 내림차순으로 정렬하고 각각의 결제 금액이 전체 결제 금액에서 어느 정도 위치하는지를 백분율로 계산하자
- 즉, 어떤 고객이 전체 고객 중 상위 몇 퍼센트에 위치하는지를 확인할 수 있음

Do it! PERCENT_RANK 함수로 상위 분포 순위를 계산

```
SELECT x.customer_id, x.amount, PERCENT_RANK() OVER (ORDER BY
x.amount DESC)
FROM (
    SELECT customer_id, sum(amount) AS amount
    FROM payment GROUP BY customer_id
) AS X
ORDER BY x.amount DESC;
```

실행 결과

	customer_id	amount	PERCENT_RANK() OVER (ORDER BY x.amount DESC)
▶	526	221.55	0
	148	216.54	0.0016722408026755853
	144	195.58	0.0033444816053511705
	137	194.61	0.005016722408026756
	178	194.61	0.005016722408026756
	459	186.62	0.008361204013377926
	469	177.60	0.010033444816053512
	468	175.61	0.011705685618729096
	236	175.58	0.013377926421404682
	181	174.66	0.015050167224080268

- 상대 순위를 계산하는 함수 — PERCENT_RANK
 - amount 열의 데이터를 내림차순하여
높은 값이 상위 몇 퍼센트인지를 계산하여 보여줌
 - 이때 가장 첫 번째 데이터는 무조건 0이 반환되며
이후 내림차순한 데이터에 따라 상위 분포를 나타냄
 - 행 마지막을 확인해 보면 1을 반환하는 것을 확인할 수 있음

▪ 첫 행 또는 마지막 행의 값을 구하는 함수 — FIRST_VALUE, LAST_VALUE

- 여기서 소개하는 함수는 그룹에서 첫 행 또는 마지막 행의 값을 구하여 데이터를 비교할 때 많이 사용함
- 예를 들어 첫 행에 있는 날짜와 매출을 기준으로 지금까지의 매출 변화를 확인할 때 사용할 수 있음
- FIRST_VALUE 함수는 그룹에서 정렬된 데이터에서 첫 번째 행의 값을, LAST_VALUE 함수는 마지막 행의 값을 반환함

- 첫 행 또는 마지막 행의 값을 구하는 함수 — FIRST_VALUE, LAST_VALUE
 - 각 함수의 기본 형식

FIRST_VALUE와 LAST_VALUE의 기본 형식

```
FIRST_VALUE(열) OVER ([PARTITION BY] ORDER BY 열)  
LAST_VALUE(열) OVER ([PARTITION BY] ORDER BY 열)
```

▪ 첫 행 또는 마지막 행의 값을 구하는 함수 — FIRST_VALUE, LAST_VALUE

- 다음은 payment 테이블에서 각 일자별로 amount 합계를 구한 다음 payment_date 열을 기준으로 오름차순 정렬하여,

FIRST_VALUE 함수를 통해 가장 낮은 일자에 대한 amount 값을 구하고,
LAST VALUE 함수를 사용하여 가장 높은 일자에 대한 amount 값을 구함

- 그리고 가장 FIRST_VALUE 값과 현재 값의 차이가 얼마인지를 조회하는 쿼리임
- 즉, 첫행을 기준으로 각 행에서의 값 차이를 구하는 것임

▪ 첫 행 또는 마지막 행의 값을 구하는 함수 — FIRST_VALUE, LAST_VALUE

Do it! FIRST_VALUE 함수로 가장 높은 값 조회

```
SELECT x.payment_date, x.amount,
       FIRST_VALUE(x.amount) OVER(ORDER BY x.payment_date) AS f_value,
       LAST_VALUE(x.amount) OVER(ORDER BY x.payment_date RANGE BETWEEN
UNBOUNDED
PRECEDING AND UNBOUNDED FOLLOWING) AS l_value,
       x.amount - FIRST_VALUE(x.amount) OVER(ORDER BY x.payment_date) AS
increase_
amount
FROM (
       SELECT date_format(payment_date, '%y-%m-%d') AS payment_d
       SUM(amount) AS amount
       FROM payment GROUP BY date_format(payment_date, '%y-%m-
) AS x
ORDER BY x.payment_date;
```

실행 결과

	payment_date	amount	f_value	l_value	increase_amount
▶	05-05-24	29.92	29.92	514.18	0.00
	05-05-25	573.63	29.92	514.18	543.71
	05-05-26	754.26	29.92	514.18	724.34
	05-05-27	684.34	29.92	514.18	654.42
	05-05-28	804.04	29.92	514.18	774.12
	05-05-29	648.46	29.92	514.18	618.54
	05-05-30	628.42	29.92	514.18	598.50
	05-05-31	700.37	29.92	514.18	670.45
	05-06-14	57.84	29.92	514.18	27.92
	05-06-15	1376.52	29.92	514.18	1346.60

▪ 첫 행 또는 마지막 행의 값을 구하는 함수 — FIRST_VALUE, LAST_VALUE

- 결과를 살펴 보면 날짜를 나타내는 payment_date 열 기준으로 오름차순에서 첫 행의 amount값(여기에선 29.92)을 찾은 다음, 각 일자별 amount 값의 차이를 보여줌
- 이러한 쿼리는 특정 행 기준으로 데이터를 비교하고 싶을 때 사용할 수 있음
- 여기서 LAST_VALUE 함수 부분을 살펴보면 RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING라는 윈도우 함수가 사용된 것을 볼 수 있음
- 이렇게 윈도우 함수를 사용하지 않으면 LAST_VALUE 함수로 정확한 결과를 얻을 수 없음